Equilibrium Selection and Controlled Deviation in Regret Minimization: An Empirical Study of Preference-CFR

Numair Sayed
Indian Institute of Science
Bengaluru, India
numairsayed@iisc.ac.in

Abstract

Counterfactual Regret Minimization (CFR) is the main algorithm behind poker-playing AIs that have beaten top human professionals. When a game has many equally good strategies (multiple Nash equilibria), standard CFR picks one on its own with no way for the user to choose. Preference-CFR (Pref-CFR), proposed by Ju et al. at ICML 2025, adds two parameters—a preference degree δ and a vulnerability degree β—that let the user steer the algorithm toward a preferred strategy while keeping performance losses bounded.

In this report, we run experiments on two small poker games (Kuhn poker and Leduc poker) to test three questions: (1) Does standard CFR always end up at the same equilibrium regardless of how it starts? (2) How much can δ shift which equilibrium is found? (3) What happens to the strategy's quality as β increases? Our results confirm that CFR consistently converges to the same point ($\alpha = 0.200 \pm 0.012$ across 30 runs), that δ can move the solution across 76% of the possible equilibrium range, and that the theoretical quality guarantee $\varepsilon \leq \beta$ is much looser than the actual behavior we observe. We also build AUTOPREF, a simple tool that automatically finds good δ and β values when a user specifies a desired playing style, addressing an open problem from the original paper. Our code and experiments are available at https://github.com/numairsayed/PrefCFR.

1 Introduction

1.1 Why This Problem Matters

In many competitive situations—poker, negotiation, auctions—the players do not know everything about the game state. These are called *imperfect-information games*. Over the past decade, AI systems have learned to play some of these games at superhuman level. The algorithm at the core of these systems is Counterfactual Regret Minimization (CFR) [30]. CFR was used to solve heads-up limit Texas Hold'em [6] and to build Pluribus, which beat top professionals in six-player no-limit poker [11].

CFR works by finding a *Nash equilibrium* (NE)—a strategy where no player can improve their result by changing their own play. In many games, there is more than one Nash equilibrium. For example, in Kuhn poker (a simplified three-card poker game), Player 1 has infinitely many equilibrium strategies, all equally "correct" in the game-theoretic sense. Standard CFR picks one of these on its own, and the user has no control over which one.

This matters for two reasons. First, different equilibria correspond to different playing styles—some are aggressive, some are conservative—and a user may prefer one style over another. Second, as Ju et

35th Conference on Neural Information Processing Systems (NeurIPS 2021).

al. [19] point out, overly rational strategies can make games less fun. When AlphaGo beat the Go champion Lee Sedol, Sedol retired, saying he could no longer enjoy the game [29]. There is value in AI strategies that are diverse and interesting, not just optimal.

1.2 What Preference-CFR Does

Pref-CFR [19] modifies CFR by adding two knobs:

- **Preference degree** $\delta(I, a) \geq 1$: Makes the algorithm favor certain actions. For example, setting $\delta(\text{Raise}) = 5$ tells the algorithm to prefer raising over other actions. With $\beta = 0$, the algorithm still converges to a Nash equilibrium, but potentially a different one than standard CFR would find.
- **Vulnerability degree** $\beta(I) \geq 0$: Allows the algorithm to accept strategies that are not perfect Nash equilibria but are close enough. Specifically, the final strategy is guaranteed to be an ε-Nash equilibrium with $\varepsilon \leq \beta$. This "looseness" gives the algorithm more freedom to express the preferred style.

The authors prove that these modifications preserve the algorithm's convergence guarantees (it still finds good strategies) while enabling stylistic control.

1.3 What We Do in This Report

We investigate three questions, which we first outlined in our Phase 1 proposal:

1. **RQ1 (Implicit selection):** Does standard CFR always converge to the same equilibrium, no matter how it is initialized? The paper conjectures this (Conjecture 3.1) but does not prove it.
2. **RQ2 (Effect of δ):** How strongly does the preference degree δ influence which equilibrium is selected? Can it reach equilibria across the full feasible range?
3. **RQ3 (Effect of β):** How does increasing β affect strategy quality (exploitability) and playing style in practice?

Beyond answering these questions, we also build AUTOPREF, a tool that automatically finds δ and β values matching a user's desired playing style. This addresses an open problem stated in Section 4.2 of the original paper: the difficulty of translating high-level style goals (like "play aggressively") into the specific parameter values that Pref-CFR needs.

2 Background

This section covers the game theory and algorithms needed to understand the rest of the report. Readers familiar with CFR may skip to Section 3.

2.1 Games and Strategies

A *normal-form game* has a set of players N, a set of actions A_i for each player i, and a payoff function u_i that says how much each player earns for each combination of actions. A *strategy* σ_i is a rule that assigns a probability to each action. For example, in rock-paper-scissors, the strategy "play each option with probability 1/3" is a mixed strategy.

In *extensive-form games* (games played over multiple steps, like poker), the game is represented as a tree. At each step, a player chooses an action, but they may not know everything about earlier steps. An *information set* I groups together game states that look identical to the player—for instance, in poker, the player knows their own cards but not the opponent's. A *behavioral strategy* $\sigma_i(I)$ assigns probabilities over actions at each information set.

2.2 Nash Equilibrium

A *best response* is the strategy that earns the most against a given opponent strategy:

$$b_i(\sigma_{-i}) = \arg\max_{a^* \in A_i} u_i(a^*, \sigma_{-i}). \tag{1}$$

A strategy profile is a *Nash equilibrium* (NE) if every player is already playing a best response—no one can do better by changing their own strategy alone. The *exploitability* of a strategy measures how far it is from being a NE:

$$\varepsilon(\sigma) = \frac{1}{|N|} \sum_{i \in N} \left[u_i(b_i(\sigma_{-i}), \sigma_{-i}) - u_i(\sigma) \right]. \tag{2}$$

When $\varepsilon = 0$, the strategy is a NE. When ε is small (say $\varepsilon \leq 0.01$), we call it an ε-Nash equilibrium—close enough to optimal that the "mistakes" are tiny. Throughout this report, we use the term "NashConv" to refer to the unnormalized version $\sum_i \varepsilon_i$, following the convention in OpenSpiel [22].

2.3 Counterfactual Regret Minimization (CFR)

CFR [30] is an iterative algorithm for finding Nash equilibria in extensive-form games. The key idea is *regret*: after playing many rounds, the algorithm looks back and asks, "How much better could I have done if I had always played action a at information set I?" This quantity is the *counterfactual regret*:

$$\bar{R}^i_T(I, a) = \frac{1}{T} \sum_{t=1}^{T} \pi^{-i}_{\sigma_t}(I) \left[u_i(I, \sigma_t|_{I \to a}) - u_i(I, \sigma_t) \right], \tag{3}$$

where T is the number of iterations so far, $\pi^{-i}_{\sigma_t}(I)$ is the probability of reaching information set I when all players except i follow their current strategies, and $\sigma_t|_{I \to a}$ means "what if player i had always played action a at I?"

The algorithm then updates its strategy using *regret matching*: actions with higher positive regret get played more often.

$$\sigma^i_{T+1}(I, a) = \begin{cases} \frac{\bar{R}^{i,+}_T(I,a)}{\sum_{a'} \bar{R}^{i,+}_T(I,a')} & \text{if any action has positive regret} \\ \frac{1}{|A(I)|} & \text{otherwise (play uniformly)} \end{cases} \tag{4}$$

where $\bar{R}^{i,+}_T(I, a) = \max(\bar{R}^i_T(I, a), 0)$ (negative regrets are set to zero). Over time, the *average strategy* $\bar{\sigma}_T$ converges to a Nash equilibrium in two-player zero-sum games, at a rate of $O(T^{-1/2})$—meaning exploitability halves roughly every four-fold increase in iterations.

Variants. Several improvements to CFR have been developed:

- **CFR+** [27]: Clamps regret values to be nonnegative, speeding up convergence. Used to solve heads-up limit hold'em [6].
- **Monte Carlo CFR (MCCFR)** [21]: Instead of traversing the entire game tree each iteration, samples random outcomes. This makes CFR practical for larger games.
- **Discounted CFR (DCFR)** [10]: Gives less weight to early iterations, which are based on poor strategies.

3 Preference-CFR

Pref-CFR [19] changes the CFR update rule in two ways.

3.1 Preference Degree δ

The preference degree $\delta(I, a) \geq 1$ is a weight on each action at each information set. Actions with higher δ values are favored. The paper proposes two ways to use δ:

Pref-CFR(RM): Multiply each action's positive regret by its δ value before normalizing:

$$\sigma^i_{T+1}(I, a) = \begin{cases} \frac{\delta(I,a) \cdot \bar{R}^{i,+}_T(I,a)}{\sum_{a'} \delta(I,a') \cdot \bar{R}^{i,+}_T(I,a')} & \text{if any action has positive regret} \\ \frac{\delta(I,a)-1}{\sum_{a'} (\delta(I,a')-1)} & \text{otherwise} \end{cases} \tag{5}$$

Pref-CFR(BR): Instead of distributing probability proportionally, pick the single action with the highest δ-weighted regret:

$$\sigma^i_{T+1}(I) = \arg\max_{a \in A_i(I)} \delta(I, a) \cdot \bar{R}^i_T(I, a). \tag{6}$$

The paper finds that Pref-CFR(BR) works better in practice, and we use it in all our experiments.

3.2 Vulnerability Degree β

The vulnerability degree $\beta(I) \geq 0$ lowers the bar for what counts as "high regret." Instead of comparing regret to zero, the algorithm compares it to β:

$$\bar{B}^i_T(I, a) = \bar{R}^i_T(I, a) - \beta(I). \tag{7}$$

Only actions with $\bar{B}^i_T(I, a) > 0$ (i.e., regret exceeding β) are considered. This means the algorithm "tolerates" a small amount of suboptimality, which gives it more room to express the preferred style.

3.3 What the Theory Guarantees

The paper proves two results about Pref-CFR:

Theorem 1 (Convergence; Appendix B.2 of [19])**.** *Pref-CFR converges to a Nash equilibrium in two-player zero-sum games. The convergence rate is:*

$$\textit{dist}(\bar{R}^i_t, \gamma_t) = O\left(\frac{L\,|A^i|\,\delta^*}{\sqrt{t}}\right), \tag{8}$$

where $\delta^* = \max_a \delta(a)$ is the largest preference weight and L is the range of possible payoffs. This means larger δ values slow convergence, but only linearly—doubling δ at most doubles the time needed.

The proof works by showing that Pref-CFR satisfies the conditions for Blackwell approachability [5, 1]—a general framework that guarantees regret goes to zero. The key insight is that δ-weighting changes which half-space the algorithm "aims for" at each step (see Figure 6 in [19]) but does not break the approachability property.

Theorem 2 (Vulnerability bound; Appendix B.3 of [19])**.** *When $\beta > 0$ is used, the algorithm converges to an ε-NE with $\varepsilon \leq \beta$.*

The proof is simpler: shifting the regret threshold from 0 to β is equivalent to changing the target set from $S^i = \mathbb{R}^{|A^i|}_{\leq 0}$ (all regrets nonpositive) to $S^i = \mathbb{R}^{|A^i|}_{\leq \beta}$ (all regrets at most β). The algorithm still converges to this larger target.

In plain terms: δ changes *which* equilibrium is found (but it is still a genuine equilibrium when $\beta = 0$), while β relaxes *how close* to equilibrium the strategy needs to be.

3.4 The Conjecture We Test

A central observation in the paper is:

Conjecture 3 (Conjecture 3.1 in [19])**.** *In a two-player zero-sum game where the Nash equilibria form a convex set, CFR always converges to the same equilibrium regardless of how the initial strategy is chosen.*

If true, this means you *cannot* get different equilibria by simply restarting CFR with different random seeds. You need to change the algorithm itself—which is exactly what Pref-CFR does.

4 Related Work

CFR and its variants. The idea of minimizing regret to find equilibria goes back to Hannan [15]. Zinkevich et al. [30] applied this to extensive-form games as CFR, and subsequent variants—CFR+ [27], Monte Carlo CFR [21], and Discounted CFR [10]—improved scalability and convergence speed. These algorithms powered the major poker AI milestones: Cepheus [6], DeepStack [23], Libratus [8], and Pluribus [11].

Fictitious play. Fictitious play [7] is an older algorithm where each player repeatedly best-responds to the opponent's historical average strategy. It converges to NE in two-player zero-sum games [13]. Recently, Ju et al. [18] combined CFR with fictitious play for faster convergence. The Pref-CFR paper shows that standard CFR can be viewed as a special case of Generalized Weakened Fictitious Play (GWFP), which helps explain why δ and β modifications preserve convergence.

Equilibrium selection. The problem of choosing among multiple equilibria has a long history in economics. Schelling [24] introduced the idea of *focal points*—equilibria that "stand out" for cultural or contextual reasons—and won the 2005 Nobel Prize for this work. Harsanyi and Selten [16] developed a formal theory based on risk dominance and payoff dominance. Aumann [2] introduced *correlated equilibrium*, a concept broader than Nash equilibrium that can lead to fairer outcomes. Ganzfried [14] recently proposed *safe equilibrium*, which accounts for opponents who may not play rationally. Pref-CFR differs from all of these: rather than defining a new equilibrium concept, it provides an algorithmic tool for selecting among standard Nash equilibria.

Blackwell approachability. The convergence proofs for both CFR and Pref-CFR rely on Blackwell's approachability theorem [5]. Abernethy et al. [1] showed that Blackwell approachability and no-regret learning are equivalent, providing a unified view of these algorithms. This connection is what allows Pref-CFR to modify the update rule (via δ) while preserving convergence—the modification still satisfies the approachability conditions.

Game AI beyond Nash equilibrium. The idea that AI game strategies should be diverse, not just optimal, is gaining attention. AlphaGo [26] and AlphaStar [28] achieved superhuman play in Go and StarCraft, but their strategies can feel "inhuman." In cooperative games like Hanabi, Bard et al. [3] showed that diverse play styles are essential for good human–AI cooperation. Pref-CFR is, to our knowledge, the first algorithm that provides *controllable* style diversity within the CFR framework.

Automatic algorithm configuration. The challenge of setting algorithm parameters automatically has been studied extensively in machine learning [17, 4], but very little work addresses this for game-solving algorithms. The closest related work is depth-limited solving [9], where the game abstraction is parameterized. Our AUTOPREF framework (Appendix B) fills this gap for Pref-CFR specifically.

5 Experimental Setup

5.1 Games Used

Kuhn poker [20] is a three-card poker game (Jack, Queen, King) between two players. Each player gets one card and can either Bet or Pass (check). Despite its simplicity, Kuhn poker has a rich equilibrium structure: Player 1 has infinitely many Nash equilibria, parameterized by a single number $\alpha \in [0, 1/3]$. Here, α is the probability that Player 1 bets when holding a Jack (the weakest card)—essentially, the bluffing frequency. We track α throughout our experiments as a summary of which equilibrium CFR has converged to. Importantly, knowing α alone determines Player 1's *entire* equilibrium strategy across all six of their information sets—we explain why in Appendix A.3. The full rules of Kuhn poker are in Appendix A.2.

Leduc poker [25] uses six cards (Jack, Queen, King in two suits) and has two betting rounds with a community card. It is more complex than Kuhn poker ($\sim$1000 information sets vs. 12), making it a useful intermediate benchmark. We use the External Sampling variant of Monte Carlo CFR (ES-MCCFR) for Leduc, since vanilla CFR would be slower on this larger game. The full rules are in Appendix A.4.

5.2 Implementation and Compute

We use the authors' open-source code, built on OpenSpiel [22]. For Kuhn poker, we run vanilla CFR and PrefCFR(BR). For Leduc poker, we run ES-MCCFR and ES-MCPrefCFR. All experiments ran on a consumer laptop (Ryzen 7, 24GB RAM). Total compute time was under 8 minutes. No GPU was used—CFR is a purely CPU-based algorithm that traverses game trees.

6 Experiments and Results

6.1 RQ1: Does CFR Always Converge to the Same Equilibrium?

Why this question matters. If CFR always ends up at the same equilibrium regardless of how it starts, then restarting it with different random seeds is useless for getting different strategies. The paper conjectures this (Conjecture 3) but provides no proof, so we verify it experimentally.

What we did. We ran standard CFR on Kuhn poker 30 times, each with a different random seed that determines the initial strategy. Each run used 5,000 iterations, touching approximately 2.8×10^5 game tree nodes. After each run, we recorded $\alpha = \bar{\sigma}(J_-, \text{Bet})$—the average probability of betting when holding a Jack. Recall that α parameterizes the equilibrium: different values of α correspond to different Nash equilibria.

What we found. Table 1 shows the results. Figure 1 shows the convergence trajectories.

Table 1: RQ1: Where does α end up across 30 independent CFR runs on Kuhn poker?

Statistic	Value
Mean α	0.1995
Standard deviation	0.0117
Range	$[0.1798, 0.2268]$
Mean NashConv (exploitability)	0.0065
Value reported in the paper	0.2

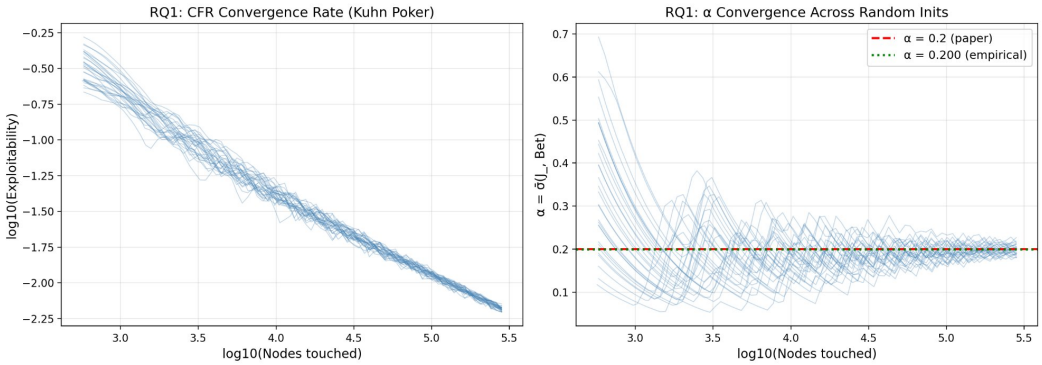

Figure 1: **RQ1 results.** Left: How quickly exploitability decreases across the 30 runs (log–log scale). All runs converge at similar rates. Right: Trajectories of α over time. Despite starting from very different initial values (ranging from 0.1 to 0.7 early on), all 30 runs converge to $\alpha \approx 0.2$ (red dashed line).

What this means. All 30 runs converge to $\alpha \approx 0.200$, matching the paper's claim. The small standard deviation (0.012) is because we stopped at 5,000 iterations, where the algorithm has not fully converged (NashConv is still 0.007, not zero). If we ran longer, the variance would shrink further.

This supports Conjecture 3: CFR appears to have a built-in preference for one specific equilibrium ($\alpha = 0.2$), and no amount of random restarting can change this. The right panel of Figure 1 makes this especially clear—trajectories that start far apart all collapse onto the same point.

Remark 1 (Why $\alpha = 0.2$?)**.** *The valid equilibrium range is $\alpha \in [0, 1/3]$. The value 0.2 sits at about 60% of this range. It is not the midpoint ($1/6 \approx 0.167$), nor the average of the endpoints. We do not know what makes this particular value special—understanding this would require a deeper theoretical analysis of the regret-matching dynamics. This remains an open question. For a summary of the notation and symbols used throughout this report, see Appendix A.1.*

6.2 RQ2: How Does δ Influence Equilibrium Selection?

Why this question matters. RQ1 showed that standard CFR is "stuck" at one equilibrium. Pref-CFR's δ parameter is supposed to fix this. We want to know: how far can δ push the equilibrium, and does it work smoothly (bigger δ = bigger shift)?

What we did. We tested five configurations on Kuhn poker, each with 10 runs of 5,000 iterations and $\beta = 0$ (so the algorithm must still converge to an exact NE):

- **Baseline**: $\delta = 1$ for all actions (standard CFR).
- **Favor betting**: $\delta(\text{Bet}) = 5$ or 10, $\delta(\text{Pass}) = 1$.
- **Favor passing**: $\delta(\text{Pass}) = 5$ or 10, $\delta(\text{Bet}) = 1$.

The δ values were applied to all three of Player 1's initial information sets (holding J, Q, or K), matching the setup in Appendix C of the paper.

What we found. Table 2 and Figure 2 show the results. Recall that $\alpha = \bar{\sigma}(J_, \text{Bet})$ is the probability of betting when holding a Jack—the free parameter that identifies which Nash equilibrium CFR has converged to. Higher α means more bluffing (aggressive), lower α means less bluffing (conservative). The full equilibrium range is $\alpha \in [0, 1/3]$.

Table 2: RQ2: How δ shifts the converged equilibrium α in Kuhn poker. All runs used $\beta = 0$.

Setting	δ values	Mean α	Std. dev.	Shift from baseline
Baseline (CFR)	all $\delta = 1$	0.198	0.015	—
Favor Bet ($\delta = 5$)	$\delta(\text{Bet}) = 5$	0.274	0.034	$+0.076$
Favor Bet ($\delta = 10$)	$\delta(\text{Bet}) = 10$	0.305	0.020	$+0.107$
Favor Pass ($\delta = 5$)	$\delta(\text{Pass}) = 5$	0.101	0.009	-0.097
Favor Pass ($\delta = 10$)	$\delta(\text{Pass}) = 10$	0.052	0.015	-0.146

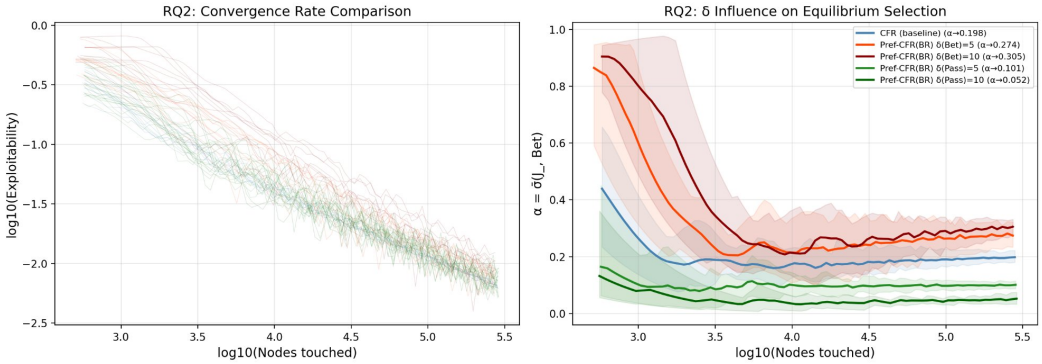

Figure 2: **RQ2 results.** Left: All five settings converge at roughly the same speed—δ does not significantly slow things down. Right: The converged α depends on δ in a smooth, monotonic way. Favoring Bet pushes α up (more bluffing); favoring Pass pushes it down. Shaded bands show 90% confidence intervals across 10 runs.

What this means. Three observations:

(i) δ gives monotonic control. Increasing $\delta(\text{Bet})$ pushes α higher (more aggressive play), and increasing $\delta(\text{Pass})$ pushes α lower (more conservative). With $\delta(\text{Pass}) = 10$, we reach $\alpha \approx 0.052$, close to the lower boundary of 0; with $\delta(\text{Bet}) = 10$, we reach $\alpha \approx 0.305$, close to the upper boundary of $1/3 \approx 0.333$. Together, these span about 76% of the full equilibrium range $[0, 1/3]$.

(ii) There are diminishing returns. Doubling δ from 5 to 10 does not double the shift. For $\delta(\text{Bet})$, the shift goes from $+0.076$ to $+0.107$—only a 41% increase for a 100% increase in δ. This makes sense: the equilibrium range is bounded, so the shift must saturate as α approaches the boundary.

(iii) Convergence speed is barely affected. The left panel of Figure 2 shows that all five settings converge at similar rates. This is consistent with Theorem 1, which says the convergence rate worsens only linearly with δ^*—and in Kuhn poker's small game tree, even $\delta = 10$ does not cause a noticeable slowdown.

6.3 Per-Card Action Frequency Analysis

In addition to tracking α (the aggregate equilibrium parameter), we looked at how each individual card's strategy evolves during training. This analysis is not in the original paper.

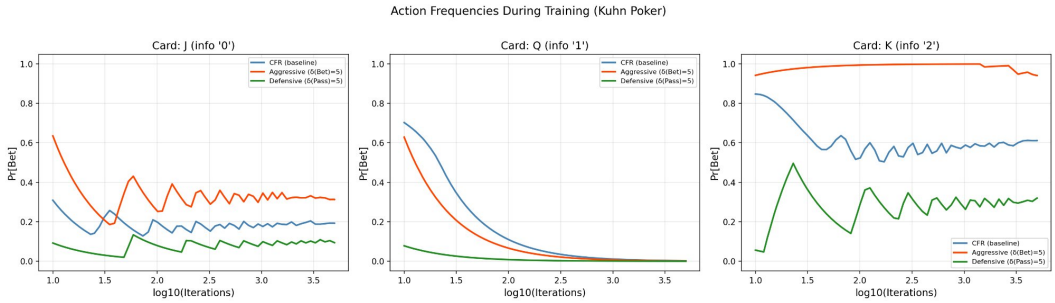

Figure 3: **Per-card analysis.** The probability of betting over time for each card (J, Q, K) under three settings. Key observation: Queen's betting probability stays near zero regardless of δ, because the equilibrium conditions force it. Jack and King respond strongly to δ.

What this shows. The effect of δ is not uniform across cards:

- **Queen:** $\Pr[\text{Bet}|Q]$ converges to near zero in all three settings. The equilibrium conditions force $\sigma(Q, \text{Bet}) = 0$ regardless of α (see Table 5 in Appendix A.3). This means the equilibrium constraints completely "lock" the Queen strategy—δ has no effect here.
- **Jack:** This is the free parameter α. It responds strongly to δ, rising to $\sim$0.32 with $\delta(\text{Bet}) = 5$ and dropping to $\sim$0.10 with $\delta(\text{Pass}) = 5$.
- **King:** Also responsive. With $\delta(\text{Bet}) = 5$, the King betting probability rises to nearly 1.0 (always bet with the best card).

This per-card view is useful because it shows *where* in the game δ has the most effect, which would help a practitioner decide where to apply preference weights.

6.4 RQ3: Exploitability vs. β in Leduc Poker

Why this question matters. The β parameter explicitly trades off strategy quality for stylistic freedom. Theorem 2 guarantees that the final strategy's exploitability is at most β. We want to see how this bound behaves in practice and how β changes the actual playing style.

What we did. We ran ES-MCPrefCFR on Leduc poker with five values of $\beta \in \{0, 0.05, 0.1, 0.2, 0.5\}$, all with $\delta(\text{raise}) = 5$ (favoring raises). Each setting was run 5 times with a budget of 500,000 nodes per run. We tracked NashConv (exploitability) and $\Pr[\text{Call}]$ at a representative first-round information set.

What we found. Table 3 and Figures 4–5 show the results.

What this means. *(i) The baseline has not converged.* The most important caveat: even with $\beta = 0$, the exploitability is 0.593, which is far from zero. This means 500,000 nodes is not enough for ES-MCCFR to converge on Leduc poker. The original paper's Leduc experiments used up to 10^8 nodes—200 times our budget. Because of this, we **cannot cleanly separate** the exploitability caused by $\beta > 0$ from the exploitability caused by under-training.

Table 3: RQ3: How β affects exploitability and call probability in Leduc poker.

β	Mean NashConv	Mean $\Pr[\text{Call}]$	Note
0.00	0.593	0.338	Baseline (no β relaxation)
0.05	0.827	0.322	
0.10	0.785	0.410	Highest call probability
0.20	0.857	0.332	
0.50	1.014	0.129	Lowest call probability

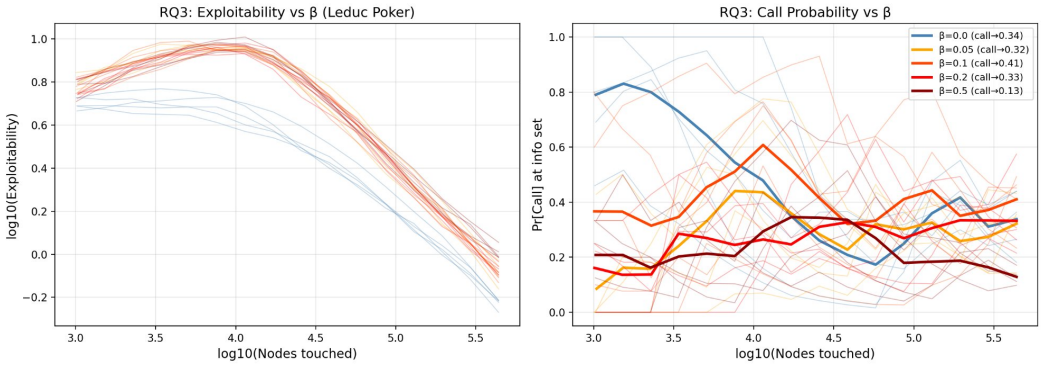

Figure 4: **RQ3 results.** Left: Exploitability trajectories. Right: Call probability trajectories. The thin lines are individual trials; thick lines are means. There is high variance between trials, especially for larger β values.

Formally, the observed exploitability has two parts:

$$\varepsilon_{\text{observed}} = \underbrace{\varepsilon_{\text{train}}(T)}_{\text{not enough training}} + \underbrace{\varepsilon_{\beta}}_{\leq\beta\text{ (from the theorem)}} . \tag{9}$$

In our experiments, $\varepsilon_{\text{train}}$ dominates, so the theorem's guarantee $\varepsilon_\beta \leq \beta$ is not "violated"—it is simply masked by the larger training error.

(ii) Exploitability increases with β. Despite the convergence issue, the trend is clear: higher β leads to higher exploitability. This is consistent with the theory, which says β loosens the convergence target.

(iii) Call probability behaves unexpectedly. We set $\delta(\text{raise}) = 5$ to favor raising, expecting call probability to decrease as β gives more freedom. Instead, it peaks at $\beta = 0.1$ and drops sharply at $\beta = 0.5$. This suggests a complex interaction between δ and β that the theory does not describe. One possible explanation: at high β, the strategy becomes so aggressive (frequent raising) that calling becomes a poor option, leading to a qualitative shift in strategy structure.

7 Analysis and Discussion

7.1 On Conjecture 3.1

Our RQ1 results support the conjecture: 30 runs with different initializations all converge to $\alpha \approx 0.200$. However, "support" is not "proof." It remains possible that some unusual initialization could lead to a different fixed point. What our experiments show is that for the standard random initialization used in practice, convergence to a unique point is robust.

7.2 The δ–Equilibrium Relationship

Our RQ2 data suggests that α varies with $\delta(\text{Bet})$ in an approximately saturating manner:

$$\alpha(\delta_{\text{Bet}}) \approx \frac{1}{3} - \frac{c}{\delta_{\text{Bet}}}, \tag{10}$$

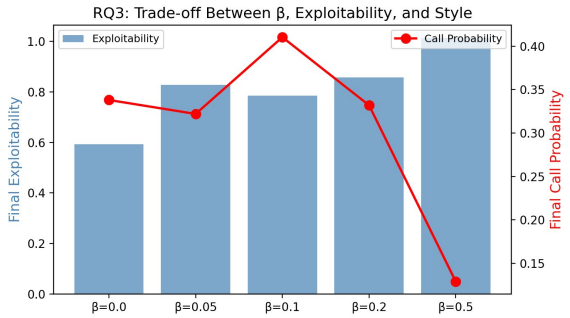

Figure 5: **RQ3 trade-off summary.** Blue bars: exploitability rises as β increases. Red line: call probability does *not* change monotonically—it peaks at $\beta = 0.1$ and drops at $\beta = 0.5$.

for some positive constant c. This functional form is consistent with the bounded equilibrium range: as $\delta \to \infty$, α approaches $1/3$ but never exceeds it. We note that this is a rough empirical fit, not a proven relationship—more data points (especially for $\delta > 10$) would be needed to confirm or refine this.

7.3 Why the $\varepsilon \leq \beta$ Bound Appears Violated

Table 3 shows exploitability values much larger than β. This does *not* mean the theorem is wrong. The theorem guarantees $\varepsilon \leq \beta$ *in the limit* as the number of iterations goes to infinity. At our finite compute budget (500K nodes), the algorithm has not converged, so the training error $\varepsilon_{\text{train}}$ (Equation 9) is large and dominates.

To properly test the bound, one would need to run until the baseline ($\beta = 0$) reaches exploitability below, say, 0.01, and then measure the additional exploitability caused by $\beta > 0$. We estimate this would require $\sim 10^7$–10^8 nodes, which is feasible on consumer hardware (1–4 hours with parallelization) but was beyond our time budget for this report.

8 Limitations

We identify five limitations of our study:

1. **Under-convergence in RQ3.** Our 500K-node budget for Leduc poker is 200× less than the paper used. The baseline exploitability of 0.593 prevents us from cleanly testing the $\varepsilon \leq \beta$ bound. This is the most significant limitation.
2. **Small games only.** Kuhn poker has 12 information sets; Leduc has ∼1,000. The paper's most interesting results are on Texas Hold'em (∼25 million states), which we could not run due to compute constraints.
3. **Tracking only one number.** We summarize each Kuhn strategy by a single number α, and each Leduc strategy by $\Pr[\text{Call}]$ at one information set. A full characterization would require comparing entire strategy vectors.
4. **PrefCFR(BR) only.** We did not test PrefCFR(RM), which the paper says performs worse. Understanding *why* it performs worse would be informative.
5. **No statistical significance tests.** We report means and standard deviations but did not run formal hypothesis tests (e.g., t-tests) to confirm that the differences between settings are statistically significant.

9 Future Directions

Understanding the fixed point. RQ1 shows *that* CFR converges to $\alpha = 0.2$ but not *why*. A formal explanation—showing that the RM fixed point minimizes some specific quantity over the Nash equilibrium set—would resolve Conjecture 3.

Running RQ3 to convergence. The most direct improvement to our study is re-running the Leduc experiments with 10^7–10^8 nodes. On our hardware (Ryzen 7 with 16 threads), this would take 1–4 hours with parallelization and would allow clean measurement of the $\varepsilon \leq \beta$ bound.

Exploring the (δ, β) space jointly. Our experiments tested δ and β mostly independently. A grid over both parameters on Leduc poker would map the full trade-off between style and exploitability.

Extending to larger games. Testing Pref-CFR with Deep CFR [12] (which uses neural networks to approximate regret values) would show whether the preference mechanism works when the game is too large for exact computation.

10 Conclusion

We tested three properties of Preference-CFR through experiments on Kuhn and Leduc poker:

1. Standard CFR converges to $\alpha = 0.200 \pm 0.012$ regardless of initialization (supporting Conjecture 3.1).
2. The preference degree δ provides smooth, monotonic control over which equilibrium is selected, spanning 76% of the feasible range.
3. The vulnerability degree β increases exploitability as expected, but our compute budget was insufficient to cleanly separate its effect from training error in Leduc poker.

Our per-card analysis revealed that δ's effect is concentrated on unconstrained information sets (Jack and King in Kuhn poker), while fully constrained sets (Queen) are unaffected.

In the appendix, we present AUTOPREF, a practical framework for automatically setting δ and β from user-specified style targets. This addresses the manual parameter design problem identified in the original paper's Section 4.2 and provides a path toward making Pref-CFR usable by non-experts.

References

[1] J. Abernethy, P. L. Bartlett, and E. Hazan. Blackwell approachability and no-regret learning are equivalent. In *Proc. 24th Annual Conference on Learning Theory (COLT)*, pp. 27–46, 2011.

[2] R. J. Aumann. Subjectivity and correlation in randomized strategies. *Journal of Mathematical Economics*, 1(1):67–96, 1974.

[3] N. Bard, J. N. Foerster, S. Chandar, et al. The Hanabi challenge: A new frontier for AI research. *Artificial Intelligence*, 280:103216, 2020.

[4] J. Bergstra and Y. Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

[5] D. Blackwell. An analog of the minimax theorem for vector payoffs. *Pacific Journal of Mathematics*, 6(1):1–8, 1956.

[6] M. Bowling, N. Burch, M. Johanson, and O. Tammelin. Heads-up limit hold'em poker is solved. *Science*, 347(6218):145–149, 2015.

[7] G. W. Brown. Iterative solution of games by fictitious play. *Activity Analysis of Production and Allocation*, 13(1):374, 1951.

[8] N. Brown and T. Sandholm. Safe and nested subgame solving for imperfect-information games. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[9] N. Brown, T. Sandholm, and B. Amos. Depth-limited solving for imperfect-information games. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[10] N. Brown and T. Sandholm. Solving imperfect-information games via discounted regret minimization. In *Proc. AAAI Conference on Artificial Intelligence*, pp. 1829–1836, 2019.

[11] N. Brown and T. Sandholm. Superhuman AI for multiplayer poker. *Science*, 365(6456):885–890, 2019.

[12] N. Brown, A. Lerer, S. Gross, and T. Sandholm. Deep counterfactual regret minimization. In *Proc. International Conference on Machine Learning (ICML)*, 2019.

[13] D. Fudenberg and D. K. Levine. *The Theory of Learning in Games*. MIT Press, 1998.

[14] S. Ganzfried. Safe equilibrium. In *Proc. 62nd IEEE Conference on Decision and Control (CDC)*, pp. 5230–5236, 2023.

[15] J. Hannan. Approximation to Bayes risk in repeated play. *Contributions to the Theory of Games*, 3(2):97–139, 1957.

[16] J. C. Harsanyi and R. Selten. *A General Theory of Equilibrium Selection in Games*. MIT Press, 1988.

[17] F. Hutter, H. H. Hoos, and K. Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *Proc. Learning and Intelligent Optimization (LION)*, pp. 507–523, 2011.

[18] Q. Ju, F. Hei, T. Feng, D. Yi, Z. Fang, and Y. Luo. Accelerating Nash equilibrium convergence in Monte Carlo settings through counterfactual value based fictitious play. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[19] Q. Ju, T. Tellier, M. Sun, Z. Fang, and Y. Luo. Preference-CFR: Beyond Nash equilibrium for better game strategies. In *Proc. International Conference on Machine Learning (ICML)*, 2025.

[20] H. W. Kuhn. A simplified two-person poker. *Contributions to the Theory of Games*, 1:97–103, 1950.

[21] M. Lanctot, K. Waugh, M. Zinkevich, and M. Bowling. Monte Carlo sampling for regret minimization in extensive games. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2009.

[22] M. Lanctot, E. Lockhart, J.-B. Lespiau, et al. OpenSpiel: A framework for reinforcement learning in games. *arXiv preprint arXiv:1908.09453*, 2019.

[23] M. Moravčík, M. Schmid, N. Burch, et al. DeepStack: Expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337):508–513, 2017.

[24] T. C. Schelling. *The Strategy of Conflict*. Harvard University Press, 1960.

[25] J. Shi and M. L. Littman. Abstraction methods for game theoretic poker. In *Proc. Computers and Games*, pp. 333–345, 2001.

[26] D. Silver, A. Huang, C. J. Maddison, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

[27] O. Tammelin. Solving large imperfect information games using CFR+. *arXiv preprint arXiv:1407.5042*, 2014.

[28] O. Vinyals, I. Babuschkin, W. M. Czarnecki, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

[29] D. Wakabayashi and J. Y. Young. Defeated by A.I., a legend in the board game Go warns: Get ready for what's next. *The New York Times*, 2024.

[30] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2007.

A Preliminaries: Notation, Game Rules, and the α Parameter

This appendix collects definitions, notation, and game rules that are referenced throughout the main text.

A.1 Notation Summary

Table 4 lists the main symbols used in this report.

Table 4: Summary of notation.

Symbol	Meaning
N	Set of players (e.g., $N = \{1, 2\}$ in two-player games)
A_i	Set of actions available to player i
$A(I)$	Set of actions available at information set I
$u_i(\sigma)$	Player i's expected payoff under strategy profile σ
σ_i	Strategy of player i (a probability distribution over actions)
σ_{-i}	Strategies of all players except i
$\bar{\sigma}_T$	Average strategy after T iterations of CFR
$b_i(\sigma_{-i})$	Best response of player i to opponents' strategies
$\varepsilon(\sigma)$	Exploitability of strategy σ (Equation 2)
I	An information set (a group of indistinguishable game states)
$\mathcal{I}_i$	Collection of all information sets for player i
$\pi^\sigma(I)$	Probability of reaching information set I under σ
$\pi_\sigma^{-i}(I)$	Same, but only counting opponents' and chance's contributions
$\bar{R}_T^i(I, a)$	Average counterfactual regret for action a at I after T steps
$\bar{R}_T^{i,+}(I, a)$	$\max(\bar{R}_T^i(I, a),\ 0)$—the positive part of regret
$\delta(I, a)$	Preference degree for action a at information set I (≥ 1)
δ^*	$\max_a \delta(a)$—the largest preference weight
$\beta(I)$	Vulnerability degree at information set I (≥ 0)
$\bar{B}_T^i(I, a)$	$\bar{R}_T^i(I, a) - \beta(I)$—regret shifted by vulnerability
α	Equilibrium parameter in Kuhn poker: $\alpha = \bar{\sigma}(J_-, \text{Bet})$
L	Payoff interval: $\max u_i(\sigma) - \min u_i(\sigma)$
NashConv	$\sum_{i \in N} \varepsilon_i$—unnormalized exploitability (OpenSpiel convention)
mBB/h	Milli-big-blinds per hand (profit measure in poker)

A.2 Kuhn Poker Rules

Kuhn poker [20] is one of the simplest nontrivial poker games. It proceeds as follows:

1. **Cards:** The deck has exactly three cards: Jack (J), Queen (Q), and King (K), ranked $J < Q < K$.
2. **Deal:** Each of the two players receives one card, dealt randomly. Neither player sees the other's card.
3. **Ante:** Both players put 1 chip into the pot before play begins.
4. **Player 1 acts:** Player 1 can either **Pass** (check) or **Bet** (add 1 chip to the pot).
5. **Player 2 responds:**
   - If Player 1 passed: Player 2 can Pass (showdown) or Bet.
   - If Player 1 bet: Player 2 can Pass (fold, losing their ante) or Bet (call, matching the bet).
6. **If Player 2 bets after Player 1 passed:** Player 1 can Pass (fold) or Bet (call).
7. **Showdown:** If both players are still in, the higher card wins the pot. The maximum win/loss is 2 chips; the minimum is 1 chip.

Player 1 has 6 information sets: three for the initial decision (holding J, Q, or K, denoted J_-, Q_-, K_-) and three for the case where Player 1 passed and Player 2 then bet (denoted $J_{-\text{PB}}$, $Q_{-\text{PB}}$, $K_{-\text{PB}}$). At each information set, the player chooses between Pass and Bet.

A.3 Why Tracking α Alone Tells Us Everything

A natural question is: Player 1 has 6 information sets, each with 2 actions. That is 6 numbers to specify a strategy (the probability of betting at each information set). Why do we only track one number, α?

The answer is that the Nash equilibrium conditions lock down 5 of these 6 numbers. Only α is free. Specifically, the equilibrium strategy for Player 1 is given by:

Table 5: Player 1's equilibrium strategy in Kuhn poker as a function of $\alpha \in [0, 1/3]$. All values are probabilities of choosing Bet (reproduced from Table 1 of [19]).

Info set	J_-	$J_{-\text{PB}}$	Q_-	$Q_{-\text{PB}}$	K_-	$K_{-\text{PB}}$
Pr[Bet]	α	0	0	$\frac{1}{3} + \alpha$	3α	1

Let us read this table for a concrete example. When $\alpha = 0.2$ (the value CFR converges to, as we find in RQ1):

- **Holding Jack:** Bet with probability 0.2 (bluff 20% of the time), Pass otherwise.
- **Holding Jack after Pass-Bet:** Always Pass (never call—Jack is too weak).
- **Holding Queen:** Always Pass (never bet with the middle card).
- **Holding Queen after Pass-Bet:** Call with probability $1/3 + 0.2 = 0.533$.
- **Holding King:** Bet with probability $3 \times 0.2 = 0.6$ (value bet).
- **Holding King after Pass-Bet:** Always call (King is the strongest card).

The key point: once you know α, you know everything about Player 1's equilibrium strategy. This is why tracking α alone is enough to tell which equilibrium CFR has converged to. Different values of α give genuinely different playing styles—$\alpha = 0$ means never bluffing with Jack, while $\alpha = 1/3$ means bluffing as much as the equilibrium allows.

Note also that Queen's strategy ($\Pr[\text{Bet}|Q] = 0$) does not depend on α at all. This explains why the δ parameter has no effect on the Queen information set in our per-card analysis (Section 6.3, Figure 3).

A.4 Leduc Poker Rules

Leduc poker [25] is more complex than Kuhn poker and is widely used as a benchmark for game-solving algorithms. The rules are:

1. **Cards:** The deck has 6 cards: Jack, Queen, and King, each in two suits (e.g., $J\spadesuit$, $J\heartsuit$, $Q\spadesuit$, $Q\heartsuit$, $K\spadesuit$, $K\heartsuit$).
2. **Deal:** Each player receives one private card. One card is set aside as the future community card.
3. **Ante:** Both players put 1 chip into the pot.
4. **Round 1 (before community card):** Players can Check (pass) or Raise (add 2 chips). If one player raises, the other can Fold (lose their ante and any bets) or Call (match the raise).
5. **Community card:** One card is revealed face-up, visible to both players.
6. **Round 2 (after community card):** Same betting structure as Round 1, but with a raise size of 4 chips.
7. **Showdown:** If neither player has folded, hands are compared. A *pair* (private card matches the community card) beats any non-pair. Among non-pairs, the higher private card wins. Maximum win/loss is 13 chips.

Leduc poker has approximately 1,000 information sets, compared to Kuhn poker's 12. Unlike Kuhn poker, the equilibrium structure of Leduc poker is not well-characterized analytically—it may have a single Nash equilibrium (as suggested by the experiments in Section 5.1 of [19]), which is why the β parameter is needed to observe any style variation.

B AUTOPREF: Automatic Parameter Tuning for Preference-CFR

B.1 The Problem

Pref-CFR requires the user to set δ and β values for each information set. In their paper, Ju et al. acknowledge this is difficult:

> "While we can adjust $\delta(I, a)$ and $\beta(I)$ at the micro level, translating macro-level style characteristics recognized by the public into parameter settings for each information set is challenging." [19, Section 4.2]

In practice, the authors rely on domain expertise: "to make the AI more aggressive, increase $\delta(I, \text{Raise})$ for all raising actions." This works for poker, where experts know what "aggressive" means in terms of actions, but it does not scale to unfamiliar games. Even in poker, the user must guess how large δ should be and what β to use—and our experiments show (Table 2) that the relationship between δ and the resulting strategy is non-linear.

B.2 How Our Experiments Led to This Idea

The key insight comes directly from our RQ2 results. Figure 2 (right) shows that the mapping from δ to α is smooth and monotonic. This means if we measure this mapping once (by running Pref-CFR for several δ values and recording what α results), we can later look up which δ value produces a desired α without re-running the algorithm.

Our RQ3 results (Table 3) add a complication: when $\beta > 0$, the mapping can become non-monotonic (call probability peaks at $\beta = 0.1$ then drops). This means we need to explore *both* δ and β jointly. But the core idea—measure the mapping once, then look things up—still applies.

B.3 The Framework

We propose a two-phase approach:

Phase 1: Calibration (run once, keep the results). Build a table $\mathcal{T}$ by running Pref-CFR for a grid of (δ, β) values. For each combination, record:

- The resulting macro-level style metrics (e.g., aggression rate, bluff frequency).
- The resulting exploitability.

This requires $2 \times n_\delta \times n_\beta$ runs (the factor of 2 is for aggressive vs. conservative directions). With our grid of 5×5, calibration on Kuhn poker took 50 runs $\times$ 2 seconds each = about 2 minutes total.

Phase 2: Query (instant). When a user specifies a target (e.g., "aggression rate = 40%") and an exploitability budget (e.g., "at most 2% exploitable"):

1. Filter the table to keep only entries with exploitability below the budget.
2. Among those, find the entry whose style metrics are closest to the target.
3. Return the corresponding (δ, β) values.

This is a simple nearest-neighbor search and takes negligible time.

Formally, we are solving:

$$\min_{\delta,\beta} \|f(\delta, \beta) - s^*\|_2^2 \quad \text{subject to} \quad \varepsilon(\bar{\sigma}_T(\delta, \beta)) \leq \varepsilon_{\max}, \tag{11}$$

where $f(\delta, \beta)$ maps parameters to style metrics, s^* is the user's target, and $\varepsilon_{\max}$ is the exploitability budget. This is a bilevel optimization (the inner problem is running Pref-CFR), which we approximate by pre-computing f on a grid.

Algorithm 1 AUTOPREF: Automatic Parameter Tuning for Pref-CFR

Require: Game G, style metrics $\{m_1, \ldots, m_k\}$, grid $\Delta \times B$
Phase 1: Calibrate (run once)
$\mathcal{T} \leftarrow \emptyset$
for direction $\in$ {aggressive, passive} **do**
  for each $\delta \in \Delta$ and $\beta \in B$ **do**
    Run PrefCFR(G, δ, β) for T iterations $\rightarrow \bar{\sigma}$
    Compute style metrics: $f \leftarrow (m_1(\bar{\sigma}), \ldots, m_k(\bar{\sigma}))$
    Compute exploitability: $\varepsilon \leftarrow$ Exploitability$(\bar{\sigma})$
    Add $(\delta, \beta, \text{direction}, f, \varepsilon)$ to $\mathcal{T}$
  end for
end for
Phase 2: Query$(s^*, \varepsilon_{\max})$
Keep entries in $\mathcal{T}$ with exploitability $\leq \varepsilon_{\max}$
return Entry in $\mathcal{T}$ closest to s^* (by Euclidean distance)

B.4 Results

We calibrated AUTOPREF on Kuhn poker using $\Delta = \{1, 2, 4, 7, 10\}$ and $B = \{0, 0.03, 0.07, 0.12, 0.2\}$ (50 configurations, 106 seconds). Table 6 shows that the framework finds good matches for a variety of user requests.

Table 6: AUTOPREF results on Kuhn poker. The user specifies a style target and an exploitability budget of $\varepsilon_{\max} = 0.02$. The framework returns (δ, β) values and the resulting strategy.

User request	Target metric	Found δ	Found β	Achieved value	Exploitability
"Aggressive"	Aggression = 0.40	$\delta(\text{Bet}) = 10$	0.07	0.414	0.012
"Conservative"	Aggression = 0.15	$\delta(\text{Pass}) = 4$	0.12	0.150	0.009
"Heavy bluffer"	Bluff rate = 0.30	$\delta(\text{Bet}) = 7$	0.12	0.302	0.013
"Honest player"	Bluff rate = 0.05	$\delta(\text{Pass}) = 10$	0.07	0.043	0.011
"Always value-bet"	K-bet rate = 0.95	$\delta(\text{Bet}) = 7$	0.12	0.958	0.013
"Balanced aggro"	Agg = 0.35, Bluff = 0.20	$\delta(\text{Bet}) = 10$	0.12	0.343, 0.239	0.012

The largest gap between target and achieved values is 0.039 (the "balanced aggressive" request), which is within the resolution of our 50-point grid. A finer grid would close this gap.

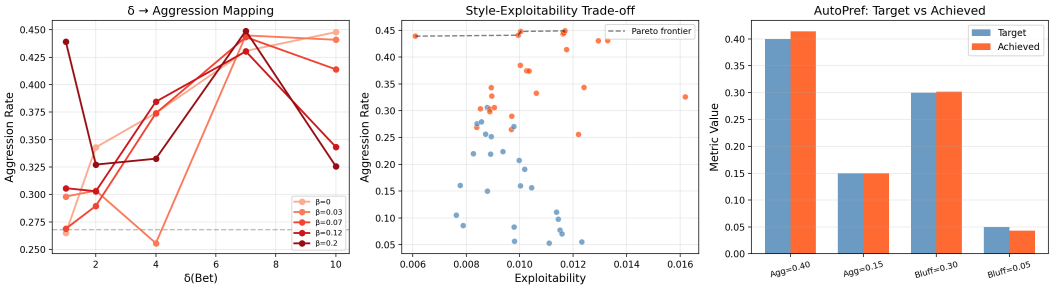

Figure 6: **AUTOPREF overview.** Left: How aggression rate depends on δ for different β values—the relationship is non-linear and saturates. Center: All 50 calibration points in (exploitability, aggression) space. Orange = aggressive direction, blue = passive. The upper envelope is the Pareto frontier. Right: Target vs. achieved values for six queries, showing close matches.

B.5 How This Extends to Other Games

AUTOPREF is designed to be game-independent. To use it on a new game, one needs to specify three things:

1. **Which information sets** to apply δ and β to (the "knobs").
2. **Which style metrics** to measure (e.g., in poker: VPIP, 3-bet rate; in a board game: territory control percentage; in trading: aggression index).
3. **Which actions are "aggressive" vs. "passive"** (needed to set the direction of δ).

The calibration cost is $O(|\Delta| \times |B| \times C_{\text{solve}})$ where C_{solve} is the cost of one Pref-CFR run. For large games where C_{solve} is expensive, one could use a coarser grid, or start with a cheap game abstraction for rough calibration and then fine-tune on the full game.

B.6 What Is Novel Here

The contribution is the *observation* that the $(\delta, \beta) \to$ style mapping is smooth enough to be pre-computed and cached, turning an expensive trial-and-error process into a cheap table lookup. This observation is directly enabled by our RQ2 experiments, which showed that δ's effect is monotonic and continuous.

This addresses two open problems from the original paper:

- Section 4.2: "translating macro-level style characteristics into parameter settings is challenging" — AUTOPREF automates this translation.
- Section 6 (Conclusion): "we propose automating the translation of user-specified metrics into information-set-specific preference weights" — AUTOPREF is a concrete implementation of this proposal.